

School Management App

Attendance Modul – Tömeges Rögzítés Minta

React 18 + Laravel 12 API | 2026

1. Miért más ez a modul, mint az eddigiek?

A Students/Teachers/Classes/Subjects modulok mind ugyanazt a mintát követték: egyesével hozunk létre, szerkesztünk, törölünk rekordokat. Az Attendance modul ETTŐL ELTÉR - itt egy OSZTÁLY ÖSSZES DIÁKJÁNAK jelenlétét egyetlen művelettel rögzítjük.

Jellemző	CRUD modulok (Students stb.)	Attendance modul
Mit ment	Egy rekordot	N diák jelenlétét egyszerre (tömb)
Form struktúra	Egyszerű mezők (name, email)	Dinamikus táblázat, soronként egy diák
State kezelés	React Hook Form	Kézi useState objektummal (attendanceMap)
Backend végpont	POST/PUT egy id-re	POST egy tömbbel (class_id + date + attendances[])

2. Az attendanceMap – objektum id szerint indexelve

A komponens állapotának szíve egy objektum, ahol a kulcsok a diák id-k, az értékek pedig a hozzájuk tartozó jelenléti adatok.

```
const [attendanceMap, setAttendanceMap] = useState({});

// Példa a futásidejű tartalomra:
{
  1: { status: 'present', note: '' },
  2: { status: 'absent', note: 'beteg' },
  3: { status: 'late', note: '' },
}
```

Miért objektum és nem tömb?

Egy tömbben (pl. [{student_id: 1, status: 'present'}, ...]) egy adott diák állapotának frissítéséhez VÉGIG kellene keresni a tömböt (find/findIndex), majd újra összeállítani. Objektummal, ahol a kulcs maga a student_id, egy adott diák állapota AZONNAL elérhető és frissíthető: attendanceMap[studentId] - nincs keresés, csak közvetlen kulcs-hozzáférés. Ez gyorsabb és egyszerűbb kód nagy diáklétszám esetén is.

2.1 Frissítés immutable módon

```
function updateStatus(studentId, status) {
  setAttendanceMap((prev) => ({
    ...prev,
    [studentId]: { ...prev[studentId], status },
  }));
}
```

Mit csinál a számított kulcs [studentId]?

A [studentId]: {...} szintaxis egy DINAMIKUS kulcsú objektum tulajdonságot hoz létre - a studentId VÁLTOZÓ értékét használja kulcsként, nem a szó szerinti 'studentId' szöveget. A ...prev előbb lemásolja az ÖSSZES meglévő diák adatát, majd a [studentId]: {...} felülírja CSAK azt az egy diákot akinek változott az állapota - a többi diák adata érintetlen marad. Ez a React state immutability (megváltoztathatatlanság) alapelve: sosem módosítjuk közvetlenül a régi state-et, mindig újat hozunk létre belőle.

3. Két adatforrás összefésülése – useEffect

A táblázat kezdeti feltöltésekor KÉT különböző forrásból érkező adatot kell összefésülni: a diákok listája (mindig létezik), és a MÁR MEGLÉVŐ jelenléti adat erre a napra (opcionális, csak ha korábban már rögzítettek valamit).

```
useEffect(() => {
  if (!students) return;

  const map = {};

  students.forEach((student) => {
    const existing = existingAttendance?.find(
      (a) => a.student?.id === student.id
    );

    map[student.id] = {
      status: existing?.status ?? 'present',
      note: existing?.note ?? '',
    };
  });

  setAttendanceMap(map);
}, [students, existingAttendance]);
```

Az összefésülés logikája - miért fontos ez a felhasználónak?

Ha egy tanár már rögzítette a jelenléte ma reggel, majd délután vissza akar térni ehhez a naphoz javítani egy elgépelést, NEM akarja hogy minden diák visszaálljon 'jelen van'-ra - a MÁR RÖGZÍTETT adatokat akarja látni és szerkeszteni. Az `existing?.find(...)` megkeresi az adott diákhoz tartozó korábbi bejegyzést, és ha van, azt használja alapértékként; ha nincs (első alkalommal rögzítünk erre a napra), az alapértelmezett 'present' (jelen van) került beállításra ?? operátorral.

Miért van MINDKÉT függőség a `useEffect` tömbjében?

A `[students, existingAttendance]` függőségi tömb azt jelenti: a `useEffect` ÚJRA lefut, ha BÁRMELYIK megváltozik. Ez szükséges, mert: 1) amikor a felhasználó ELŐSZÖR választ osztályt, a `students` érkezik meg előbb; 2) amikor DÁTUMOT vált, az `existingAttendance` frissül egy már kiválasztott osztálynál. Mindkét esetben újra kell építeni az `attendanceMap`-et a friss adatokkal.

4. Adatátalakítás mentéskor – objektum vissza tömbbé

A backend egy TÖMBÖT vár (`attendances: [{student_id, status, note}, ...]`), de a komponens állapota egy OBJEKTUM (`attendanceMap`). Mentéskor ezt vissza kell alakítani.

```
const attendances = Object.entries(attendanceMap).map(
  ([studentId, data]) => ({
    student_id: Number(studentId),
    status: data.status,
    note: data.note || null,
  })
);

await recordAttendance.mutateAsync({
  class_id: Number(classId),
  date,
  attendances,
});
```

Mit csinál az `Object.entries()`?

Az `Object.entries({1: {...}, 2: {...}})` egy TÖMBBÉ alakítja az objektumot, ahol minden elem egy [kulcs, érték] pár: `[[1, {status: 'present'}], [2, {status: 'absent'}]]`. A `map()`-ben ezt destruktráljuk (`[studentId, data]`) és átalakítjuk a backend által várt `{ student_id, status, note }` formára. A `Number(studentId)` fontos, mert az objektum kulcsai MINDIG stringek JavaScript-ben (még ha számokkal indexeljük is őket), a backend viszont integer `student_id`-t vár.

5. React Query hook-ok az Attendance modulhoz

```
// Meglévő jelenlét lekérdezése egy adott napra
export function useClassAttendance(classId, date) {
  return useQuery({
    queryKey: ['attendances', 'class', classId, date],
    queryFn: () => api.get(`/attendances/class/${classId}`,
      { params: { date } }).then((res) => res.data.data),
    enabled: !!classId && date,
  });
}

// Tömeges mentés
export function useRecordStudentAttendance() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: (payload) =>
      api.post('/attendances/students', payload).then((res) =>
        res.data),
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['attendances'] });
    },
  });
}
```

Miért enabled: !!classId && date)?

Ez a lekérdezés KÉT paramétertől függ, nem csak egytől. A !!classId && date) csak akkor ad true-t, ha MINDKETTŐ jelen van (classId nem üres string, date nem üres). Ha a felhasználó még nem választott osztályt (classId === ''), a lekérdezés NEM fut le - elkerülve egy hibás /attendances/class/ (üres id-vel) kérést.

6. Összefoglaló – mikor válasszuk ezt a mintát?

Válaszd EZT a mintát (tömeges), ha:	Válaszd a SZOKÁSOS CRUD mintát, ha:
Sok hasonló rekordot egyszerre kell módosítani	Egy önálló entitást hozunk létre/szerkesztünk
A rekordok egy közös kontextushoz tartoznak (egy nap, egy osztály)	A rekordoknak nincs közös csoportosító kontextusuk
A felhasználó egy táblázatban akar áttekinteni és módosítani mindent	Egy dedikált form oldal elegendő

Előretekintés - hol lesz még szükség erre a mintára?

Az Exams modulban a vizsgaeredmények rögzítése (ExamResultsPage) UGYANEZT a mintát fogja követni: egy vizsgához tartozó ÖSSZES diák eredményét egyszerre rögzítjük egy táblázatban, objektum-alapú state-tel és tömbbé alakítással mentéskor. A most megtanult minta közvetlenül újrafelhasználható lesz ott.